

---

# Predictive Analytics (cont.)

Evaluation metrics, and the traps inside them

**Prof. Gabriel Spadon, PhD**

Faculty of Computer Science  
spadon@dal.ca

# 1 Metrics for classification

The same five predictions can be reported as “5% error”, as “60% of a naïve baseline”, as  $r^2 = 0.79$  (squared correlation) or as  $R^2 = 0.65$  (residual definition). All four are true. The metric is part of the claim.

This section works the classification metrics on one six-case example. The regression sections that follow make the same argument with numeric targets.

## 1.1 The confusion matrix and accuracy

**What it is for** A confusion matrix records every prediction a classifier made on a test set. Every classification metric in this class is computed from it.

**The definition** A confusion matrix is a square  $n_c \times n_c$  table, where  $n_c$  is the number of class values of the problem. Each entry counts how many times the pair (*ObservedClass*, *PredictedClass*) occurred on the test cases. Observed values index the rows and predicted values the columns (Table 1).

**Table 1.** General form of an  $n_c \times n_c$  confusion matrix

|      |       | Pred.     |           |           |
|------|-------|-----------|-----------|-----------|
|      |       | $c_1$     | $c_2$     | $c_3$     |
| Obs. | $c_1$ | $n_{1,1}$ | $n_{1,2}$ | $n_{1,3}$ |
|      | $c_2$ | $n_{2,1}$ | $n_{2,2}$ | $n_{2,3}$ |
|      | $c_3$ | $n_{3,1}$ | $n_{3,2}$ | $n_{3,3}$ |

**The formula** The diagonal holds the correct predictions and everything off it is an error, so

$$accuracy = \frac{n_{1,1} + n_{2,2} + n_{3,3}}{N}, \quad error\ rate = 1 - accuracy.$$

The error rate follows entirely from the counts in the table.

## 1.2 Cost-sensitive classification

**What it is for** Error rate gives unit cost to every mistake and zero cost to every correct prediction. When mistakes differ in consequence, a cost matrix states how much each one costs.

**The rule** Missing a fraudulent transaction and flagging a legitimate one do not cost the same. A second matrix of the same shape as the confusion matrix gives the benefit  $B_{i,i}$  of each correct prediction and the cost  $C_{i,j}$  (a negative benefit) of each type of error (Table 2). Models are then compared on **utility**, the sum of the benefits minus the costs of all predictions, rather than on a count of mistakes.

## 1.3 Precision and recall

**What it is for** When one class is rare, accuracy is dominated by the majority class. With 99% negatives, a model that never predicts the positive class scores 99% accuracy and is useless. Precision and recall look only at the positive class.

**Table 2.** Cost/benefit matrix with the same shape as the confusion matrix

|             |       | Pred.     |           |           |
|-------------|-------|-----------|-----------|-----------|
|             |       | $c_1$     | $c_2$     | $c_3$     |
| <b>Obs.</b> | $c_1$ | $B_{1,1}$ | $C_{1,2}$ | $C_{1,3}$ |
|             | $c_2$ | $C_{2,1}$ | $B_{2,2}$ | $C_{2,3}$ |
|             | $c_3$ | $C_{3,1}$ | $C_{3,2}$ | $B_{3,3}$ |

**Table 3.** Confusion matrix cells for a binary classifier

|             |   | Preds. |    |
|-------------|---|--------|----|
|             |   | P      | N  |
| <b>Obs.</b> | P | TP     | FN |
|             | N | FP     | TN |

**The formulas** For a binary classifier the four cells are named as in Table 3.

$$Prec = \frac{TP}{TP + FP}, \quad Rec = \frac{TP}{TP + FN}.$$

Precision is the proportion of the model's signals (events) that are correct. Recall is the proportion of the real events that the model captures.

**Worked example** Six test cases give the confusion matrix in Table 4, with  $TP = 2$ ,  $FN = 2$ ,  $FP = 1$  and  $TN = 1$ .

**Table 4.** Confusion matrix for the six-case example

|             |   | Preds. |   |
|-------------|---|--------|---|
|             |   | P      | N |
| <b>Obs.</b> | P | 2      | 2 |
|             | N | 1      | 1 |

$$Precision = \frac{2}{2+1} = 0.667, \quad Recall = \frac{2}{2+2} = 0.5, \quad ErrorRate = \frac{2+1}{2+2+1+1} = 0.5.$$

**The trade-off** Either metric is easy to improve at the cost of the other. Predict P for all six cases and recall rises to 1 while precision stays at  $4/6 = 2/3$ . Predict P only for the single case the model is most confident about and precision can reach 1 while recall falls to 0.25. Recall alone is trivial to maximize, so neither number means anything without the other.

## 1.4 The $F$ -score

**What it is for** Selecting a model on two metrics at once is hard. The  $F$ -score combines precision and recall into one number.

**The formula**

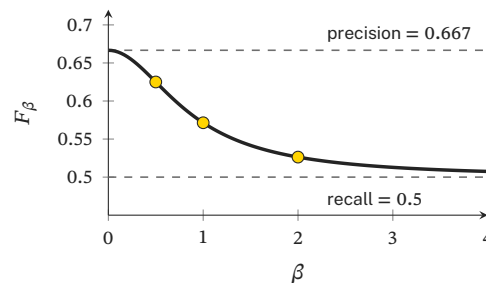
$$F_{\beta} = \frac{(\beta^2 + 1) \cdot Prec \cdot Rec}{\beta^2 \cdot Prec + Rec}$$

$F_1$  is the harmonic mean of precision and recall, and  $F_\beta$  is a weighted harmonic mean, van Rijsbergen's effectiveness measure.[7] The parameter  $\beta$  is user-defined and defaults to 1. As  $\beta \rightarrow 0$  the weight of recall decreases and  $F_\beta \rightarrow \text{Prec}$ . As  $\beta \rightarrow \infty$  the weight of precision decreases and  $F_\beta \rightarrow \text{Rec}$ .

**Worked example** On the six cases ( $\text{Prec} = 2/3$ ,  $\text{Rec} = 1/2$ ),

$$F_{0.5} = \frac{1.25 \cdot \frac{2}{3} \cdot \frac{1}{2}}{0.25 \cdot \frac{2}{3} + \frac{1}{2}} = 0.625, \quad F_1 = \frac{2 \cdot \frac{2}{3} \cdot \frac{1}{2}}{\frac{2}{3} + \frac{1}{2}} = \frac{4}{7} = 0.571, \quad F_2 = \frac{5 \cdot \frac{2}{3} \cdot \frac{1}{2}}{4 \cdot \frac{2}{3} + \frac{1}{2}} = 0.526.$$

Figure 1 traces  $F_\beta$  as  $\beta$  grows. It starts at precision and falls towards recall.



**Figure 1.**  $F_\beta$  against  $\beta$  for  $\text{Prec} = 0.667$  and  $\text{Rec} = 0.5$

**How to read  $\beta$**  Choosing  $\beta$  states a preference between precision and recall. Recall has relative weight  $\beta^2$  in the weighted harmonic mean. Equivalently,  $F_\beta = (1 + \beta^2)TP / [(1 + \beta^2)TP + \beta^2FN + FP]$ . This is not a fixed operational cost ratio, and it does not replace a cost/benefit matrix.

## R

```
obs <- factor(c("P","P","P","P","N","N"))
pred <- factor(c("P","P","N","N","P","N"))
cm <- caret::confusionMatrix(data = pred,
                             reference = obs,
                             positive = "P")
cm$byClass[c("Precision", "Recall", "F1")]
# 0.6666667 0.5000000 0.5714286
```

## Python

```
from sklearn.metrics import (precision_score,
                             recall_score, f1_score, fbeta_score)
obs = ["P", "P", "P", "P", "N", "N"]
pred = ["P", "P", "N", "N", "P", "N"]
precision_score(obs, pred, pos_label="P")
# 0.6666666666666666
recall_score(obs, pred, pos_label="P") # 0.5
f1_score(obs, pred, pos_label="P")
# 0.5714285714285715
fbeta_score(obs, pred, beta=2, pos_label="P")
# 0.5263157894736842
```

**Differences to know.** `caret::confusionMatrix` prints *Prediction* down the rows and *Reference* across the columns, which is the transpose of the layout above. It treats the *first factor level* as the positive class unless you pass `positive=`. R orders factor levels alphabetically by default, so "N" would be positive here, changing which class the class-specific metrics describe. `sklearn.metrics.confusion_matrix` puts true labels on the rows and predicted on the columns in sorted label order, so its top-left cell is TN rather than TP. Both give the same three numbers once the positive class is stated. For undefined precision, scikit-learn returns 0.0 with an `UndefinedMetricWarning` unless you set `zero_division`; caret returns NA.

## 2 The regression test set

**What it is for** Every regression metric below is computed on this one table, so the metrics can be compared number for number.

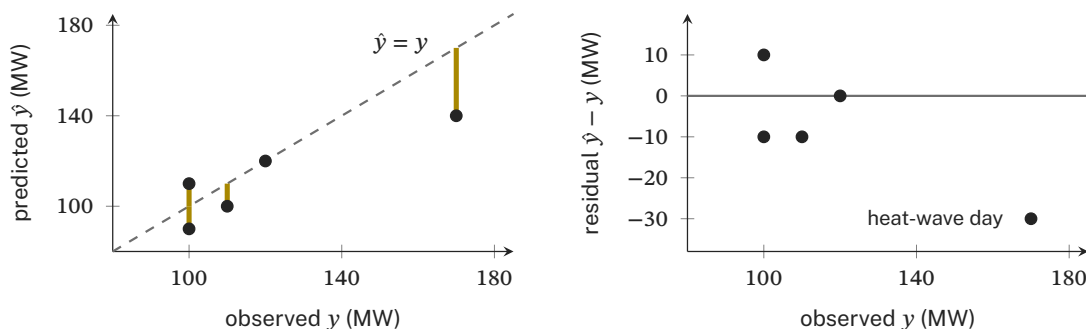
**The data** The test set is five held-out days (Table 5). The target  $Y$  is next-day peak electrical load in MW, and  $\hat{y}_i$  is the model's prediction.

**Table 5.** Worked regression example over five held-out days

| $i$      | $y_i$ | $\hat{y}_i$ | $e_i = \hat{y}_i - y_i$ | $ e_i $   | $e_i^2$     | $ \bar{y} - y_i $ | $(\bar{y} - y_i)^2$ |
|----------|-------|-------------|-------------------------|-----------|-------------|-------------------|---------------------|
| 1        | 100   | 110         | +10                     | 10        | 100         | 20                | 400                 |
| 2        | 110   | 100         | -10                     | 10        | 100         | 10                | 100                 |
| 3        | 100   | 90          | -10                     | 10        | 100         | 20                | 400                 |
| 4        | 120   | 120         | 0                       | 0         | 0           | 0                 | 0                   |
| 5        | 170   | 140         | -30                     | 30        | 900         | 50                | 2500                |
| $\Sigma$ | 600   | 560         | -40                     | <b>60</b> | <b>1200</b> | <b>100</b>        | <b>3400</b>         |

So  $N_{test} = 5$  and  $\bar{y} = 600/5 = 120$  MW. Three rows are there to test specific metrics.

- Rows 1 and 2 swap the same two numbers ( $y = 100, \hat{y} = 110$  and  $y = 110, \hat{y} = 100$ ). They test MAPE.
- Rows 1 and 3 hold  $y = 100$  and move the prediction up and down by 10 ( $\hat{y} = 110$  and  $\hat{y} = 90$ ). They test sMAPE.
- Row 4 is an exact prediction, so it adds a zero term to every error metric.
- Row 5 is a heat-wave day on which the model under-predicts by 30 MW. It is the largest error.



**Figure 2.** Predicted versus observed values and residuals for the test days

### Look before you average

Draw both plots in Figure 2 before computing any metric. The predicted-versus-actual plot shows whether the predictions sit on the line  $\hat{y} = y$  or on some other line. The residual plot shows whether the errors are patternless or grow with the target. Here there are four errors of at most 10 MW and one of 30 MW. Each metric below is a different answer to how much that fifth point should count.

### 3 Mean squared error and its root

**What it is for** MSE is the standard criterion for fitting a regression model. RMSE is the same quantity in the units of the target, for reporting.

**The formulas**

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (\hat{y}_i - y_i)^2, \quad RMSE = \sqrt{MSE},$$

where  $\hat{y}_i$  is the prediction of the model under evaluation for case  $i$  and  $y_i$  is the true target value.

**Worked example** From the  $e_i^2$  column of Table 5,

$$MSE = \frac{100 + 100 + 100 + 0 + 900}{5} = \frac{1200}{5} = 240 \text{ MW}^2, \quad RMSE = \sqrt{240} = 15.49 \text{ MW}.$$

**Properties**

- **Units.** MSE is in the square of the target's units,  $\text{MW}^2$  here and dollars squared for a price model. Squared units are hard to interpret operationally. RMSE restores the units of  $Y$ .
- **Outliers.** MSE is very sensitive to them because it punishes large errors more. The single 30 MW error contributes 900 of the 1200, so 20% of the cases produce 75% of the score.
- **Differentiable.** MSE is differentiable everywhere, which makes it usable as the criterion a model is fitted to optimize. Least squares is exactly the choice of MSE as the training objective.
- **Same ranking.** The square root is monotone, so RMSE ranks two models on the same test set exactly as MSE does. MSE is commonly used for fitting and RMSE for reporting.

### 4 Mean absolute error, and choosing between MAE and RMSE

**What it is for** MAE reports the typical size of an error, with every unit of error counted once.

**The formula**

$$MAE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} |\hat{y}_i - y_i|.$$

**Worked example** From the  $|e_i|$  column,

$$MAE = \frac{10 + 10 + 10 + 0 + 30}{5} = \frac{60}{5} = 12 \text{ MW}.$$

**Properties** MAE is in the same units as  $Y$ . It is more robust to outliers than MSE, because an error of 30 counts three times an error of 10 rather than nine times. It is not differentiable at  $y_i = \hat{y}_i$ , which makes it harder to use as the criterion to optimize a model.

## 4.1 What each one answers

Which of the two to report is a published debate.[1, 12] MAE answers *on a typical day, how far off am I?* RMSE answers *how exposed am I to a bad day?* Whether sensitivity to outliers is good or bad depends on the application. If the operational cost of an error grows faster than linearly (a grid failure, an overdraft, a stock-out), sensitivity is what you want and RMSE is the right metric. If the large errors are recording mistakes or one-off events you cannot forecast, RMSE measures your data quality rather than your model.

**Worked example** Repair the worst day only. Move  $\hat{y}_5$  from 140 to 160, so the largest error falls from 30 MW to 10 MW and every other row is untouched. Table 6 recomputes both metrics.

**Table 6.** MAE and RMSE before and after reducing the largest error

|      | before   | after   | change |
|------|----------|---------|--------|
| MAE  | 12.00 MW | 8.00 MW | −33%   |
| RMSE | 15.49 MW | 8.94 MW | −42%   |

MAE falls by 33% and RMSE by 42%, a larger drop, because RMSE was mostly reporting that one day. Two facts make this general.

- $MAE \leq RMSE \leq \sqrt{N_{test}} \cdot MAE$  always. The left equality holds only when every error has the same magnitude, the right one when at most one case has nonzero error. Here  $RMSE/MAE = 15.49/12 = 1.29$  sits between 1 and  $\sqrt{5} = 2.24$ , and its distance from 1 measures how uneven the errors are.
- MAE is minimized by predicting the conditional *median* of  $Y$ . MSE and RMSE are minimized by predicting the conditional *mean*. Choosing a metric therefore chooses which summary of the predictive distribution a fitted model will chase.

Report both. The gap between them shows how much the largest errors dominate.

### R

```
y <- c(100, 110, 100, 120, 170)
p <- c(110, 100, 90, 120, 140)

mean(abs(p - y))      # MAE 12
mean((p - y)^2)       # MSE 240
sqrt(mean((p - y)^2)) # RMSE 15.49193

library(Metrics)
mae(y, p); mse(y, p); rmse(y, p) # identical

caret::postResample(pred = p, obs = y)
#      RMSE Rsquared    MAE
# 15.491933 0.7949126 12.000000
```

### Python

```
import numpy as np
from sklearn.metrics import (mean_absolute_error,
                             mean_squared_error, root_mean_squared_error,
                             r2_score)

y = np.array([100, 110, 100, 120, 170])
p = np.array([110, 100, 90, 120, 140])

mean_absolute_error(y, p) # 12.0
mean_squared_error(y, p) # 240.0
root_mean_squared_error(y, p) # 15.49193384829668
np.sqrt(np.mean((p - y) ** 2)) # 15.49193384829668
r2_score(y, p) # 0.6470588235294117
```

### Differences to know, three defaults

1. **RMSE in scikit-learn.** `mean_squared_error(y, p, squared=False)` used to be the way to get RMSE. The `squared` argument was deprecated in scikit-learn 1.4 and *removed* in 1.6. On a current install the call raises `TypeError: got an unexpected keyword argument 'squared'`. Use `root_mean_squared_error`, added in 1.4.
2.  **$n$  or  $n-1$ .** None of MSE, RMSE or MAE divides by  $n-1$ . They are plain means. Normalizing by a variance brings the difference back. R's `var()` divides by  $n-1$  and NumPy's `np.var()` divides

by  $n$ . On this data  $\text{MSE}/\text{var}(y) = 240/850 = 0.282$  in R and  $240/\text{np.var}(y) = 240/680 = 0.353$  in NumPy. Only the second equals the NMSE of section 5.

3. **Two different  $R^2$ .** `caret::postResample` prints `Rsquared = 0.795` above, while `r2_score` prints 0.647 for the same vectors. Neither is a bug (section 6).

### Differences to know, missing values

R propagates NA silently. `mean(abs(p - y))` on a vector with one missing prediction returns NA, not an error. scikit-learn refuses outright (`ValueError: Input contains NaN`), and NumPy returns nan unless you call `np.nanmean`. Decide whether a missing prediction is an error to be scored or a case to be dropped, and say which in the write-up. Dropping it changes  $N_{test}$ , and therefore every metric in this handout.

## 5 NMSE and NMAE

**What it is for** Whether 12 MW is a good error depends on the scale and difficulty of the problem. A relative metric divides the model's error by the error of a stated baseline, so it can be compared across data sets with different scales.

**The formulas** With a baseline that always predicts the mean  $\bar{y}$ ,

$$NMSE = \frac{\sum_{i=1}^{N_{test}} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N_{test}} (\bar{y} - y_i)^2}, \quad NMAE = \frac{\sum_{i=1}^{N_{test}} |\hat{y}_i - y_i|}{\sum_{i=1}^{N_{test}} |\bar{y} - y_i|}.$$

Both are unitless. Below 1 the model beats the baseline, at 1 it ties, and above 1 it loses. Both need a nonzero denominator, so a constant test target makes them undefined.

**Worked example** Take  $\bar{y} = 120$ , the mean of the test targets. The last two columns of Table 5 give the denominators.

$$NMSE = \frac{1200}{3400} = 0.353, \quad NMAE = \frac{60}{100} = 0.600.$$

The model makes 35% of the baseline's squared error and 60% of its absolute error. The two differ for the same reason RMSE and MAE differ. The squared version is dominated by the heat-wave day, on which the baseline is also badly wrong.

### 5.1 Say which baseline

A training-set mean is a predictor you could actually deploy. The test-set mean is a retrospective reference, and it is the one residual  $R^2$  uses. The two give different scores for the same predictions on the same test set. Table 7 supposes the training mean had been 130 MW.

Neither choice is wrong. Reporting the number without saying which one you used is. The same holds for any "normalized", "relative" or "skill" score, because a normalized metric is a comparison and needs a named competitor.



**Table 7.** NMSE under the test-mean and training-mean baselines

| baseline $\bar{y}$ | $\sum(\bar{y} - y_i)^2$ | NMSE  |
|--------------------|-------------------------|-------|
| test mean, 120     | 3400                    | 0.353 |
| training mean, 130 | 3900                    | 0.308 |

## 6 $R^2$ and the squared correlation

**What it is for**  $R^2$  is the most widely reported regression score. Two different quantities carry the name, and software reports both.[6]

### 6.1 Residual $R^2$ , and why it can be negative

**The formula** With the test mean as the baseline,  $R^2$  is the complement of NMSE.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 1 - NMSE = 1 - \frac{1200}{3400} = 0.647.$$

**Negative values** For ordinary least squares with an intercept,  $R^2$  on the training data lies in  $[0, 1]$  when the target is nonconstant, because least squares can do at least as well as predicting the mean. On a test set that guarantee disappears, because  $\bar{y}$  is not something the model was allowed to tune. A model that predicts a constant 100 MW for all five days has  $SS_{res} = 0 + 100 + 0 + 400 + 4900 = 5400 > 3400$ , so

$$R^2 = 1 - \frac{5400}{3400} = -0.588.$$

A negative  $R^2$  means the model is worse than predicting the mean. It is a valid, reportable result.

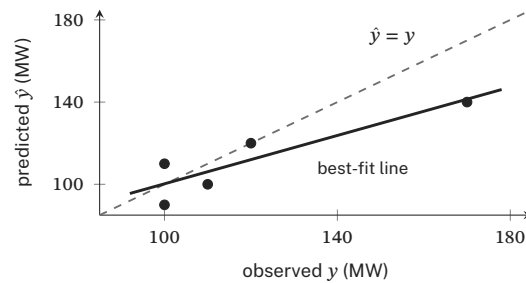
### 6.2 Squared correlation

**The formula** The second quantity called “R squared” is the square of Pearson’s correlation between  $y$  and  $\hat{y}$ .

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 0.647, \quad r^2 = \left( \frac{\sum(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum(y_i - \bar{y})^2 \sum(\hat{y}_i - \bar{\hat{y}})^2}} \right)^2 = 0.795.$$

**Worked example** Here  $\bar{\hat{y}} = 112$ ,  $\sum(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}}) = 2000$ ,  $\sum(y_i - \bar{y})^2 = 3400$  and  $\sum(\hat{y}_i - \bar{\hat{y}})^2 = 1480$ , so  $r = 2000/\sqrt{3400 \times 1480} = 0.8916$  and  $r^2 = 0.7949$ .

**Why they differ** The best-fit line through the predicted-versus-actual scatter has slope 0.588 and intercept 41.41 MW, not slope 1 and intercept 0 (Figure 3). The predictions compress the range. Squared correlation measures linear association and does not change when  $\hat{y}$  is shifted or rescaled, so it cannot see this bias.  $R^2$  compares against the line  $\hat{y} = y$  and does see it.



**Figure 3.** The test points follow a line of slope 0.59, not the line  $\hat{y} = y$

### When the two agree

$R^2 = r^2$  is guaranteed when  $\hat{y}$  holds nonconstant ordinary least-squares fitted values from a linear model *with an intercept*, scored on the same data used to fit it. Most students first meet  $R^2$  in that setting. Refit a line of  $y$  on  $\hat{y}$  here ( $y = -31.35 + 1.351 \hat{y}$ ) and its  $R^2$  is 0.7949, exactly  $r^2$ . That calibration step is what the raw predictions were missing. On held-out data the equality is not guaranteed, and here the squared correlation is larger by about 0.15. Fitting the calibration on the test set illustrates the identity. It is not a valid held-out performance estimate.

### R

```
caret::R2(p, y, form = "traditional")
# 0.6470588 1 - SSres/SStot
caret::R2(p, y) # 0.7949126 cor(p, y)^2,
# the default form
cor(y, p)^2 # 0.7949126
1 - sum((y - p)^2) / sum((y - mean(y))^2)
# 0.6470588
```

### Python

```
from sklearn.metrics import r2_score
r2_score(y, p)
# 0.6470588235294117 always 1 - SSres/SStot
np.corrcoef(y, p)[0, 1] ** 2 # 0.7949125596184420
r2_score(y, np.full(5, 100))
# -0.5882352941176470 worse than the mean
```

**Differences to know.** `caret::postResample` and `caret::defaultSummary` (the default regression summary in `caret`) report the *correlation* version. `sklearn.metrics.r2_score` reports the *residual* version and has no option to do otherwise. A model comparison table that mixes the two is not a comparison. State which definition you used, and prefer the residual one out of sample, because it is the one that detects bias.

## 7 MAPE

**What it is for** MAPE expresses each error as a fraction of the true value. The result is scale-free and easy to explain to non-specialists.[4]

### The formula

$$MAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|\hat{y}_i - y_i|}{y_i}$$

**Worked example** The five terms are 0.100, 0.0909, 0.100, 0 and 0.1765, so

$$MAPE = \frac{0.4674}{5} = 0.0935 = 9.35\%.$$

## Properties

- **Undefined if the true value is zero.** A nonzero error divided by a target approaching zero grows without bound.
- **Not differentiable** at  $y_i = \hat{y}_i$ . Optimizing it needs a method that handles this nonsmooth point.
- **Needs a meaningful zero.** The targets here are positive, on a scale where zero means none. A percentage error on a temperature in degrees Celsius is meaningless, because that zero is a convention.
- **Asymmetric.** Exchanging  $y_i$  and  $\hat{y}_i$  gives a different MAPE. In rows 1 and 2,  $y_i = 100$ ,  $\hat{y}_i = 110$  scores 0.1, while  $y_i = 110$ ,  $\hat{y}_i = 100$  scores 0.09.
- **Ranges from 0 to  $\infty$ .** There is no upper limit when the predictions are too high, but it cannot exceed 100% for under-predictions constrained to be nonnegative.

### Reading the asymmetry correctly

MAPE is asymmetric when truth and prediction exchange roles. It does not penalize over-prediction more than an equal-sized under-prediction at fixed truth.

With the true value *held fixed*, MAPE is symmetric in the sign of the error. For  $y = 100$ , both  $\hat{y} = 110$  and  $\hat{y} = 90$  score  $10/100 = 0.1$ , because the denominator is the truth and the truth did not change.

Swapping the *roles* of the two numbers changes the denominator. The pair  $\{100, 110\}$  scores  $10/100 = 0.1$  when the smaller number is the truth and  $10/110 = 0.09$  when the larger one is. *For the same absolute error, MAPE is larger when the true value is smaller.*

Because  $\hat{y} \geq 0$ , an under-prediction has  $|\hat{y} - y|/y \leq 1$ , so it never costs more than 100%, while an over-prediction is unbounded. MAPE is absolute error weighted by  $1/y$ , so small targets count more. Optimizing it can favor lower forecasts than optimizing MAE or MSE.[9] It does not encode a specified operational cost of under-forecasting.

### The zero, and what the software does with it

Add one quiet hour to the test set with  $y_6 = 0$  and  $\hat{y}_6 = 5$ . The sixth term is  $5/0$ , so the metric is undefined, and direct floating-point division returns  $\infty$ . One row destroys the metric. Table 8 shows how each library reports it.

**Table 8.** R and Python results for MAPE with a zero denominator

| call                                                              | result    |
|-------------------------------------------------------------------|-----------|
| <code>mean(abs((y - p) / y))</code> in R                          | Inf       |
| <code>Metrics::mape(y, p)</code>                                  | Inf       |
| <code>sklearn.metrics.mean_absolute_percentage_error(y, p)</code> | 3.753e+15 |

scikit-learn replaces  $y_i$  by  $\max(|y_i|, \epsilon)$  with  $\epsilon \approx 2.2 \times 10^{-16}$ . The result is finite and enormous, with no exception and no warning, and is easy to mistake for a real score. Check for zeros in the target before you report a percentage error, in either language.

## 8 sMAPE

**What it is for** sMAPE puts the prediction into the denominator alongside the truth, to remove MAPE's role-swap asymmetry.

### The formula

$$sMAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|\hat{y}_i - y_i|}{|\hat{y}_i| + |y_i|}$$

**Worked example** The five terms are 10/210, 10/210, 10/190, 0 and 30/310, so

$$sMAPE = \frac{0.2446}{5} = 0.0489 = 4.89\%.$$

### Properties

- **Role-swap symmetric.** Exchanging  $y_i$  and  $\hat{y}_i$  leaves the value unchanged. Rows 1 and 2 both score  $10/210 = 4.76\%$ .
- **Asymmetric in another way.** Hold the true value and the absolute deviation fixed and move in the two directions. Rows 1 and 3 give  $y_i = 100$ ,  $\hat{y}_i = 110$  at  $4.76\%$  and  $y_i = 100$ ,  $\hat{y}_i = 90$  at  $10/190 = 5.26\%$ . The denominator grows when the prediction is high and shrinks when it is low.[2]
- **Bounded but fragile near zero.** The value lies in  $[0, 1]$ . It is undefined when  $y_i = \hat{y}_i = 0$  and sensitive to small changes when both are near zero.
- **Needs a meaningful zero**, as MAPE does.

### Differences to know, two sMAPE conventions

The denominator above is  $|\hat{y}_i| + |y_i|$ , which bounds the metric in  $[0, 1]$ , that is 0–100%. The other widely used definition divides by  $(|\hat{y}_i| + |y_i|)/2$  and therefore reports exactly *twice* the value, on a 0–200% scale. R's `Metrics::smape` uses that doubled form.[4] A “sMAPE of 9.8%” and a “sMAPE of 4.89%” can describe the same predictions. Check the definition inside the function you call before comparing your number with a published one.

The code below computes the relative metrics of sections 5–8.

### R

```
sum((p - y)^2) / sum((mean(y) - y)^2)
# NMSE 0.3529412
sum(abs(p - y)) / sum(abs(mean(y) - y))
# NMAE 0.6

library(Metrics)
mape(y, p)          # 0.09347594
smape(y, p)         # doubled convention: 0.09785755

mae <- mean(abs(p - y))
mse <- mean((p - y)^2)
rmse <- sqrt(mse)
c(mae = mae, mse = mse, rmse = rmse,
  mape = mape(y, p),
  nmse = 1 - caret::R2(p, y, form = "traditional"),
  nmae = sum(abs(p - y)) / sum(abs(mean(y) - y)))
```

### Python

```
from sklearn.metrics import (r2_score,
                             mean_absolute_percentage_error)

1 - r2_score(y, p)          # NMSE 0.3529411764705883
mean_absolute_percentage_error(y, p)
# 0.09347593582887701 (a fraction)

# scikit-learn has no NMAE and no sMAPE;
# write them out
np.sum(np.abs(p - y)) / np.sum(np.abs(y.mean() - y))
# 0.6
np.mean(np.abs(p - y) / (np.abs(p) + np.abs(y)))
# 0.04892877
```

**Differences to know.** The explicit R formulas normalize `nmse` and `nmae` against the mean of the values passed to them. If the training mean is the intended baseline, compute and substitute it explicitly. `1 - r2_score()` always uses the test mean. `sklearn`'s `MAPE` returns a fraction, not a percentage, so multiply by 100 before putting it in a table.

## 9 Nine numbers from one test set

**What it is for** This section puts every regression metric side by side, so the choice between them is visible (Table 9).

**Table 9.** Nine regression metrics computed on the same test set

| Metric                        | Value  | Units           | What it answers                                             |
|-------------------------------|--------|-----------------|-------------------------------------------------------------|
| MAE                           | 12.00  | MW              | Typical size of an error.                                   |
| MSE                           | 240.00 | MW <sup>2</sup> | Average squared error; fitting criterion.                   |
| RMSE                          | 15.49  | MW              | Typical error, with large errors weighted up.               |
| NMSE                          | 0.353  | —               | Squared error as a fraction of the baseline's.              |
| NMAE                          | 0.600  | —               | Absolute error as a fraction of the baseline's.             |
| $R^2 = 1 - SS_{res}/SS_{tot}$ | 0.647  | —               | Improvement over the test-mean baseline; = $1 - NMSE$ here. |
| $r^2$ (squared correlation)   | 0.795  | —               | How well predictions track, ignoring bias.                  |
| MAPE                          | 9.35%  | %               | Error relative to each true value.                          |
| sMAPE                         | 4.89%  | %               | Error relative to truth and prediction together.            |

One model and one test set give nine defensible numbers. A reader told “about 5% off” and a reader told “60% of a naïve baseline” form very different impressions of the same predictions. Choose the metric before you see the results, justify it from the decision the model supports, and report it with its baseline, its units and the size of the test set.

## 10 Common misconceptions

- **“RMSE and MAE measure the same thing in different units.”** They agree only when all errors have the same magnitude. The ratio  $RMSE/MAE$  measures how uneven the errors are, and it ranges from 1 to  $\sqrt{N_{test}}$ .
- **“ $R^2$  is between 0 and 1.”** This is guaranteed in-sample for ordinary least squares with an intercept and a nonconstant target. On a test set  $R^2$  can be arbitrarily negative, and a negative value is a result, not a fault.
- **“ $R^2$  is the square of the correlation.”** This is guaranteed for nonconstant ordinary least-squares fitted values with an intercept on the same data. Here the two differ by 0.15, and two popular libraries report different ones by default.
- **“A high correlation means good predictions.”** Correlation does not change under shifts and positive rescaling, so predictions that are all 50 MW too high, or all half the right size, can still correlate at 0.99.
- **“MAPE is scale-free, so it is always safe.”** It needs a strictly positive scale with a meaningful zero, it is undefined at  $y = 0$ , and its inverse-target weighting can favor low forecasts.
- **“sMAPE fixes MAPE's asymmetry.”** It fixes the role-swap asymmetry and introduces another one, and it comes in two conventions that differ by a factor of two.

- **“NMSE below 1 means the model is good.”** It means the model beat a constant baseline, on this test set, with that baseline’s mean. Against a seasonal or last-value baseline the same model may lose.
- **“Accuracy is a reasonable default for classification.”** Not when the classes are imbalanced. That is why precision, recall and  $F_\beta$  exist.

## 11 Summary

---

- Each metric has its own pros and cons.
- If the data set is imbalanced, accuracy is misleading. Use precision and recall, or other metrics that do not overly focus on the majority class (sometimes called the negative class). Selecting a model on two metrics at the same time may be difficult, so use  $F_1$ , the harmonic mean of precision and recall, or its weighted version  $F_\beta$ .
- MSE is differentiable, which makes it easier to use as a model optimization criterion.
- RMSE and MAE are expressed in the same units as the target variable.
- Relative metrics make it easier to compare results across data sets with different scales.
- Name the baseline of a normalized metric, say which  $R^2$  your software reported, and check percentage-metric denominators for zeros.

## References

---

- [1] T. Chai and R. R. Draxler. “Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature”. In: *Geoscientific Model Development* 7 (2014), pp. 1247–1250.
- [2] P. Goodwin and R. Lawton. “On the asymmetry of the symmetric MAPE”. In: *International Journal of Forecasting* 15.4 (1999), pp. 405–408.
- [3] B. Hamner and M. Frasco. *Metrics: Evaluation Metrics for Machine Learning*. CRAN.
- [4] R. J. Hyndman and A. B. Koehler. “Another look at measures of forecast accuracy”. In: *International Journal of Forecasting* 22 (2006), pp. 679–688.
- [5] M. Kuhn. *caret: Classification and Regression Training*. Version 7.0-1. Package documentation for model training, resampling, tuning and evaluation. CRAN. 2024.
- [6] T. O. Kvålseth. “Cautionary note about  $R^2$ ”. In: *The American Statistician* 39.4 (1985), pp. 279–285.
- [7] C. J. van Rijsbergen. *Information Retrieval*. 2nd ed. London: Butterworths, 1979.
- [8] scikit-learn developers. *Metrics and Scoring: Quantifying the Quality of Predictions*. User guide, with mean\_absolute\_error, root\_mean\_squared\_error, r2\_score and mean\_absolute\_percentage\_error. URL: [https://scikit-learn.org/stable/modules/model\\_evaluation.html](https://scikit-learn.org/stable/modules/model_evaluation.html).
- [9] C. Tofallis. “A better measure of relative prediction accuracy for model selection and model estimation”. In: *Journal of the Operational Research Society* 66.8 (2015), pp. 1352–1362.
- [10] L. Torgo. *Data Mining with R: Learning with Case Studies*. 2nd ed. CRC Press, 2016.
- [11] L. Torgo. *performanceEstimation: An Infra-Structure for Performance Estimation of Predictive Models*. Version 1.1.0. Archived on CRAN in 2025. CRAN. 2016.

- [12] C. J. Willmott and K. Matsuura. “Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance”. In: *Climate Research* 30 (2005), pp. 79–82.